Project Instructions

Introduction to Robotic Manipulation

Contents

Project Overview **2**

Hardware and Software **2**

General Project Workflow **3**

Phase 1: Kinematic Modelling **3**

Phase 2: Inverse Kinematics **4**

Phase 3:- Cartesian Control and Final Demonstration **4**

Required Demonstration Task **5**

Hardware Testing **5**

Deliverables **6**

Deliverable 1: Code Repository 6

Deliverable 2: Technical Report 7

Deliverable 3: Final Demonstration 8

Project Overview

The final project brings together the main concepts covered throughout the course **Introduction to Robotic Manipulation**. Students will work in teams of **2–3** to model, simulate, control, and experimentally demonstrate a physical **LeRobot SO-101 robotic arm**.

The primary objective is to develop an **inverse-kinematics-based Cartesian control system** that enables the robot end-effector to move to desired positions or poses.

The project integrates:

- Forward kinematics
- Inverse kinematics
- Manipulator Jacobians
- Numerical optimization
- Joint-limit handling
- Singularity awareness
- Trajectory generation
- Robot control
- Simulation
- Hardware implementation

The project accounts for **50 % of the final course grade.**

Hardware and Software

Hardware

Each team will be assigned a **LeRobot SO-101 robotic arm**.

The physical robot should only be used after the algorithms have been successfully tested in simulation. Lab access for hardware testing is planned **as posted on Moodle**.

Software

The main software environment for the project is:

- Python 3.12
- NumPy, SciPy, matplotlib
- Miniforge
- MuJoCo

- MATLAB
- LeRobot SDK
- Git

MuJoCo will be used for simulation and initial validation, while the LeRobot SDK will provide the interface to the physical robot.

General Project Workflow

The project should follow the sequence:

Physical Robot → Kinematic Model → Forward Kinematics → Simulation Validation → Inverse Kinematics → Trajectory Generation → Simulated Control → Hardware Deployment → Evaluation

Do not directly command untested Inverse Kinematics solutions to the physical robot.

Important: The controller should first be validated in simulation. Only after it behaves correctly and safely in MuJoCo should it be deployed on the physical robot.

Phase 1: Kinematic Modelling

The objective of Phase 1 is to obtain an accurate mathematical model of the LeRobot SO-101.

Students will:

- Measure the relevant physical dimensions of the robot.
- Define the robot coordinate frames.
- Derive either a **Denavit–Hartenberg (DH)** or **Product of Exponentials (PoE)** representation.
- Implement forward kinematics.
- Validate the forward kinematics against measured or simulated end-effector positions.
- Document the resulting kinematic model.

Forward Kinematics Implementation

Implement a function that computes the end-effector transformation from the joint configuration:

$$T_{0e} = f(\mathbf{q}), \tag{1}$$

where

$$\mathbf{q} = \begin{bmatrix} q_1 & q_2 & \cdots & q_n \end{bmatrix}^T . \tag{2}$$

The resulting transformation should provide both the end-effector position

$$\mathbf{p} = \begin{bmatrix} x \\ y \\ z \end{bmatrix} , \tag{3}$$

and orientation

$$R \in SO(3). \tag{4}$$

A suitable software interface could be:

```
T = forward_kinematics(q)
```

or

```
position, rotation = forward_kinematics(q)
```

Forward kinematics should be validated using several robot configurations.

Do not proceed to inverse kinematics until the forward-kinematics model has been verified.

Phase 2: Inverse Kinematics

The objective of Phase 2 is to develop an algorithm that determines the joint configuration required to achieve a desired Cartesian end-effector pose.

This phase requires:

- Numerical IK using either the **Jacobian pseudoinverse** or **damped least squares**.
- Handling of robot joint limits.
- Consideration of singular or poorly conditioned configurations.
- Validation in simulation before hardware deployment.

Phase 3:- Cartesian Control and Final Demonstration

The final phase integrates the IK implementation with robot control.

Students are required to:

- Move the end-effector toward commanded Cartesian positions or poses.
- Generate smooth trajectories between targets or waypoints.
- Demonstrate the system by tracing a predefined geometric shape or following a sequence of Cartesian waypoints.

Cartesian Pose Commands

The final system should allow a target Cartesian position or pose to be specified, for example:

```
target_position = [x, y, z]
```

or

```
target_pose = T_desired
```

The controller should:

1. Determine the required joint configuration using IK.
2. Generate a suitable trajectory between the current and desired configurations.
3. Command the simulated or physical robot.
4. Determine whether the desired target has been reached.

Required Demonstration Task

The final controller must demonstrate Cartesian end-effector motion along a predefined path. Choose one of the following options.

Option A: Geometric Shape

Trace a predefined shape such as:

- Square
- Rectangle
- Triangle
- Circle

Option B: Waypoint Sequence

Follow a predefined sequence of Cartesian waypoints:

$$P_1 \rightarrow P_2 \rightarrow P_3 \rightarrow \cdots \rightarrow P_N. \tag{5}$$

The trajectory should require meaningful motion of the robot and demonstrate that the controller operates in **Cartesian space**, rather than simply replaying manually specified joint configurations.

Hardware Testing

When deploying the controller on the physical LeRobot:

1. Begin from a known safe configuration.
2. Test small Cartesian motions first.
3. Use conservative motion speeds.
4. Keep targets within the verified robot workspace.
5. Check joint limits before sending commands.
6. Be prepared to stop the robot immediately if unexpected motion occurs.
7. Do not deliberately command configurations that may cause self-collision or collision with the environment.

Note: Behaviour observed in simulation may not transfer perfectly to the physical robot. Differences due to modelling errors, servo behaviour, calibration, and hardware limitations should be expected.

Deliverables

Each team must submit the following **three deliverables**.

Deliverable 1: Code Repository

Submit a **Code repository** containing the complete project implementation.

The source code should be clearly organized and documented.

A recommended structure is:

```
lerobot_ik_project/
|
|-- README.md
|-- requirements.txt
|
|-- src/
|   |-- forward_kinematics.py
|   |-- jacobian.py
|   |-- inverse_kinematics.py
|   |-- trajectory.py
|   `-- robot_control.py
|
|-- simulation/
|   `-- mujoco_demo.py
|
|-- hardware/
```

```
|   `-- lerobot_demo.py
|
|-- tests/
|   |-- test_fk.py
|   `-- test_ik.py
|
`-- results/
    |-- figures/
    `-- data/
```

The exact structure may differ, but another student or instructor should be able to understand and run the project without significant additional guidance.

README Requirements

The README should include:

- Project title
- Team members
- Brief project description
- Installation/setup instructions (if any)
- Instructions for running the MuJoCo simulation
- Instructions for running the final demonstration
- Brief description of the main files

The repository should provide sufficiently clear instructions for reproducing the project results.

Deliverable 2: Technical Report

Submit a **4–6 page technical report** describing the development and evaluation of the system.

The report should cover:

1. **Kinematic model**
    - Robot coordinate frames
    - DH or PoE representation
    - Forward-kinematics derivation and validation
2. **Inverse kinematics**
    - Selected numerical IK method
    - Jacobian computation
    - Convergence criteria
    - Joint-limit and singularity handling

3. **Controller and trajectory generation**
    - Cartesian command implementation
    - Trajectory-generation method
    - Controller implementation and relevant parameters
4. **Results and discussion**
    - Simulation results
    - Hardware results
    - Position or trajectory accuracy
    - Limitations
    - Failure cases
    - Possible improvements

Deliverable 3: Final Demonstration

Each team must provide either:

- a **live demonstration**, or
- a **recorded video demonstration**,

showing the physical LeRobot SO-101 executing the final Cartesian control task.

The demonstration should clearly show:

- The physical robot
- The selected geometric path or waypoint sequence
- The robot executing the commanded motion
- Successful use of the implemented IK-based Cartesian controller

The final demonstration should use the team's own kinematic model, IK implementation, trajectory generation, and robot-control code.